Neurobehavioural Adaptive Robot

Complete Build Plan for a Biosignal-Driven Interactive Robot Platform

Substrate House
Neuroscience · Engineering · Data Science
London, UK

April 2026 · v1.0

Abstract

This document presents a complete build plan for a physically embodied robot that reads real-time multimodal biosignal data from a human participant, passes it through a neurobehavioural foundation model, and adapts its behaviour based on the inferred cognitive and physiological state. The plan covers robot form factor, hardware components, compute architecture, software stack, real-time data pipeline, behaviour design, and a phased build schedule with milestones. The target is a working prototype within 14 weeks.

Contents

1 System Overview **3**

2 Robot Form Factor and Design Rationale **3**
2.1 Recommended Form: Mobile Tabletop Companion . . . 3
2.2 Design Rationale . . . 3

3 Hardware Specification **4**
3.1 Compute Hardware . . . 4
3.2 Mobile Base . . . 4
3.3 Expressive Head . . . 5
3.4 Biosignal Acquisition (Wearable on Human) . . . 5
3.5 Power System . . . 6
3.6 Cost Summary . . . 6

4 Compute Architecture **6**
4.1 Architecture Diagram . . . 6
4.2 Model Optimisation for Edge Deployment . . . 7

5 Software and Programming Stack **8**
5.1 Stack Overview . . . 8
5.2 Key Libraries . . . 8
5.3 ROS2 Node Architecture . . . 9

6 Real-Time Data Pipeline **9**
6.1 End-to-End Latency Budget . . . 9
6.2 Stream Synchronisation . . . 9
6.3 Signal Processing per Modality . . . 10

7 Behaviour Design **10**
7.1 State-to-Behaviour Mapping . . . 10
7.2 Behaviour Primitives . . . 11
7.3 Policy Logic . . . 11

8 Build Order and Milestones **12**
8.1 Phase 0: Software Foundations (Weeks 1–2) . . . 13
8.2 Phase 1: Robot Base (Weeks 3–4) . . . 13
8.3 Phase 2: Expressive Head (Weeks 5–7) . . . 13
8.4 Phase 3: Multi-Modal Input (Weeks 8–10) . . . 13
8.5 Phase 4: Polish and Optimise (Weeks 11–14) . . . 14

9 Risks and Mitigations **14**

10 Data Clarification **15**

11 Future Directions **15**

A Communication Topology **16**

1 System Overview

The system consists of three principal subsystems: (1) wearable biosignal sensors on the human, (2) an edge compute unit running the foundation model, and (3) a physically embodied robot that adapts its behaviour in real time. Figure 1 shows the high-level architecture.

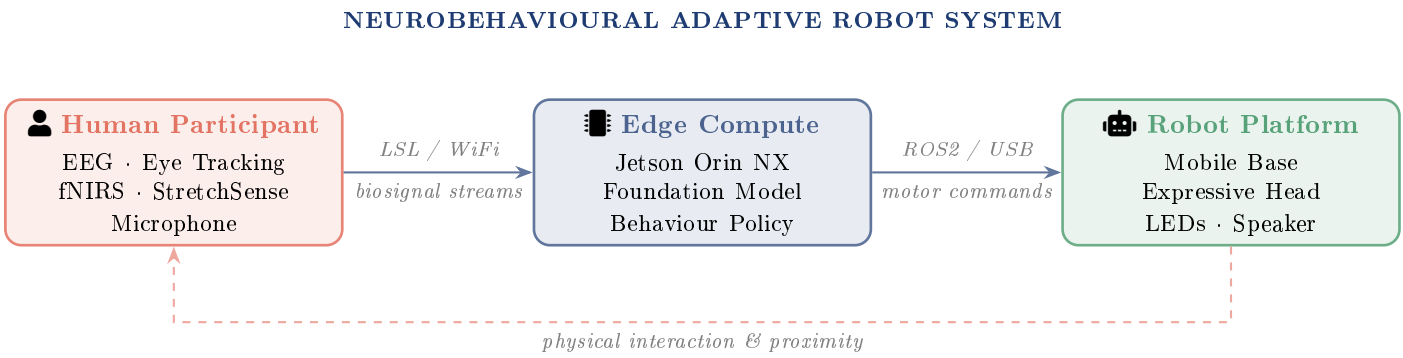

Figure 1: High-level system architecture. Biosignal data flows from the human through edge compute to the robot; the robot's physical behaviour feeds back to the human through the interaction loop.

2 Robot Form Factor and Design Rationale

2.1 Recommended Form: Mobile Tabletop Companion

The primary output channel is **behavioural adaptation**—pace, proximity, expression, movement style—not physical manipulation. This dictates a mobile companion form factor rather than an arm or humanoid.

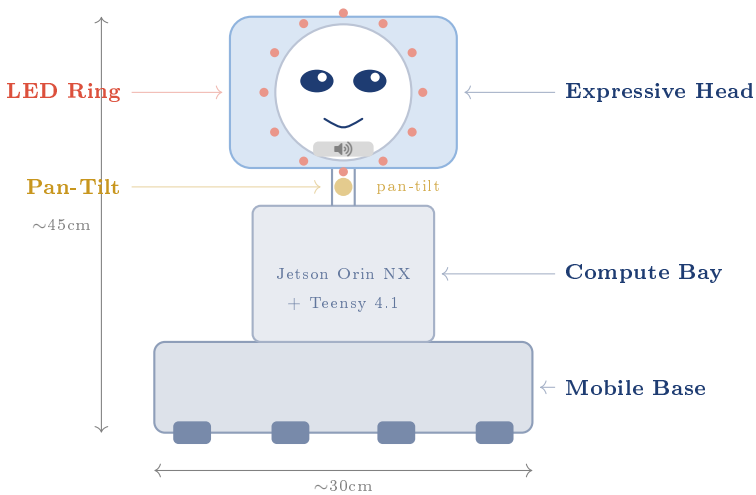

Figure 2: Robot form factor concept. A mobile base supports a compute bay and an expressive pan-tilt head with round display face, LED ring, and speaker. Total height approximately 45 cm.

2.2 Design Rationale

- **Mobile base** enables proximity modulation—the robot can approach or retreat based on arousal and stress levels.

- **Expressive head** with pan-tilt provides the fastest, most legible feedback channel: gaze direction, expression changes, and head orientation communicate social signals that humans read instinctively.
- **Compact tabletop scale** (∼30–50 cm) keeps the build mechanically simple and safe for close interaction.
- **No arms in v1**—manipulation adds 10× complexity with minimal benefit for adaptive behaviour output.

3 Hardware Specification

3.1 Compute Hardware

Table 1: Compute components.

Component	Part	Cost (£)	Role
Edge AI Computer	NVIDIA Jetson Orin NX 16 GB	450–550	Foundation model inference (PyTorch, TensorRT, ∼100 TOPS INT8)
Carrier Board	Seeed reComputer J4012	incl.	Jetson carrier with I/O breakout
Microcontroller	Teensy 4.1 (600 MHz Cortex-M7)	25	Real-time motor control, sensor polling, safety watchdog
Offload Server (opt.)	Linux PC + RTX 4090	varies	If model exceeds Jetson capacity

3.2 Mobile Base

Table 2: Mobile base options.

Component	Part	Notes
Base Platform	Robotis TurtleBot4 Lite (£1,100)	ROS2-native out of box; saves 2–3 months vs custom
Alt: Custom Chassis	Aluminium 4WD (∼30×25 cm)	Cheaper but requires more integration
Alt: Motors	4× JGB37-520 DC geared (12 V, encoders)	Encoders essential for odometry
Motor Driver	Cytron MDD10A dual H-bridge	Simple, reliable
Robot IMU	BNO055 9-DOF	Robot orientation and acceleration
LiDAR (v2)	RPLidar A1M8	SLAM and navigation

3.3 Expressive Head

Table 3: Head and expression components.

Component	Part	Notes
Pan-Tilt Servos	Dynamixel XL330-M288-T (×2)	Excellent small servos; Dynamixel SDK has ROS2 support
Servo Controller	Dynamixel U2D2	USB-to-Dynamixel interface
Face Display	Waveshare 5" round LCD	Renders animated eyes and expressions
Speaker	3 W speaker + MAX98357A I2S amp	Tones, simple vocalisations
LED Ring	NeoPixel WS2812B (24-LED)	Ambient state indicator
Housing	3D-printed PLA/PETG	Custom head shell

3.4 Biosignal Acquisition (Wearable on Human)

Table 4: Biosignal hardware and streaming protocols.

Stream	Hardware	Protocol
EEG	mBrainTrain Smarting PRO (24 ch) *or* OpenBCI Cyton+Daisy (16 ch)	LSL (native)
Eye Tracking	Pupil Labs Neon *or* Tobii Pro Glasses 3	LSL (plugin)
fNIRS	NIRx NIRSport2 (wireless)	LSL (native)
StretchSense	Existing StretchSense gloves	Custom UDP → LSL relay
Microphone	Rode Wireless GO II	ALSA → LSL
Screen Recording	OBS Studio + NDI	LSL timestamp markers

3.5 Power System

Table 5: Power components.

Component	Specification
Robot Battery	12 V 5 Ah LiPo (motors and base)
Jetson Power	7–20 V input, ∼15 W typical / 25 W peak; Pololu 12 V-to-19 V step-up from robot battery
Battery Management	Low-voltage cutoff; monitored via Teensy ADC

3.6 Cost Summary

Table 6: Estimated total cost (excluding biosignal hardware already owned).

Item	Est. Cost (£)
Jetson Orin NX 16 GB + carrier board	550
TurtleBot4 Lite	1,100
Dynamixel XL330-M288-T (×2) + U2D2	120
Waveshare 5" round display	45
NeoPixel 24-LED ring	10
Speaker + MAX98357A amp	15
Teensy 4.1	25
12 V LiPo battery + charger	80
DC-DC converter (Pololu)	15
3D printing filament	30
Cables, connectors, misc	50
Total	**∼£2,040**

4 Compute Architecture

4.1 Architecture Diagram

Figure 3 shows the full compute pipeline from biosignal acquisition through model inference to motor command generation.

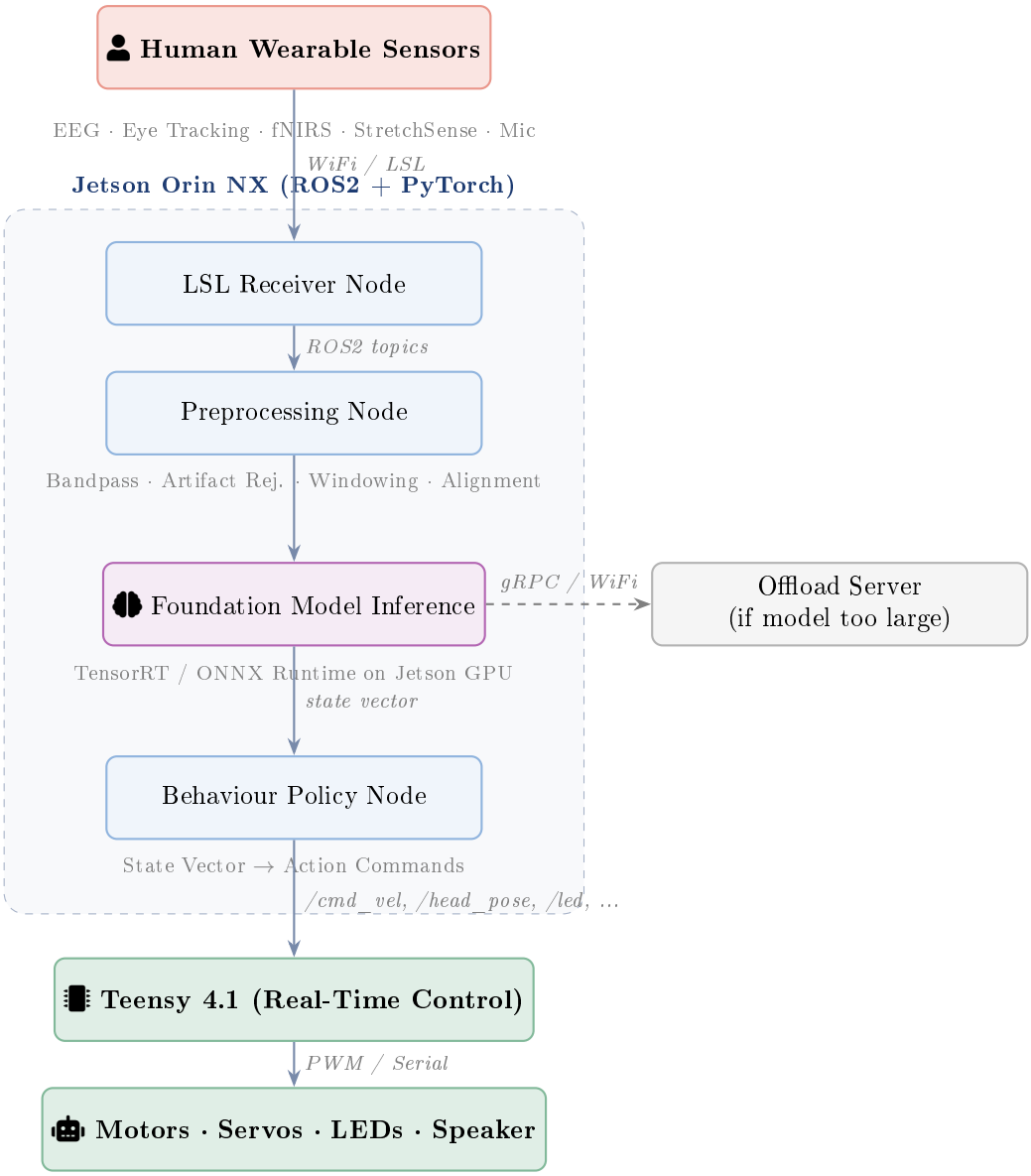

Figure 3: Compute architecture. All ROS2 nodes run on the Jetson Orin NX. The Teensy handles hard real-time motor control. An optional offload server handles inference if the model exceeds Jetson capacity.

4.2 Model Optimisation for Edge Deployment

The foundation model must run inference in **<200 ms per window** to feel responsive:

1. **Export to ONNX**, then optimise with **TensorRT** on Jetson—expect 2–5× speedup over vanilla PyTorch.
2. **Quantise to FP16 or INT8**. For biosignal embeddings, FP16 is almost always lossless.
3. If the model exceeds ∼500M parameters, **offload to a server** over WiFi. Budget ∼20–50 ms network round-trip on local 5 GHz WiFi.
4. Target: **5–10 Hz inference rate** (100–200 ms per cycle).

> **Note**
>
> Design the inference node to support both local and remote execution from day one. A simple configuration flag can switch between `LocalInferenceBackend` and `RemoteInferenceBackend` (gRPC client).

5 Software and Programming Stack

5.1 Stack Overview

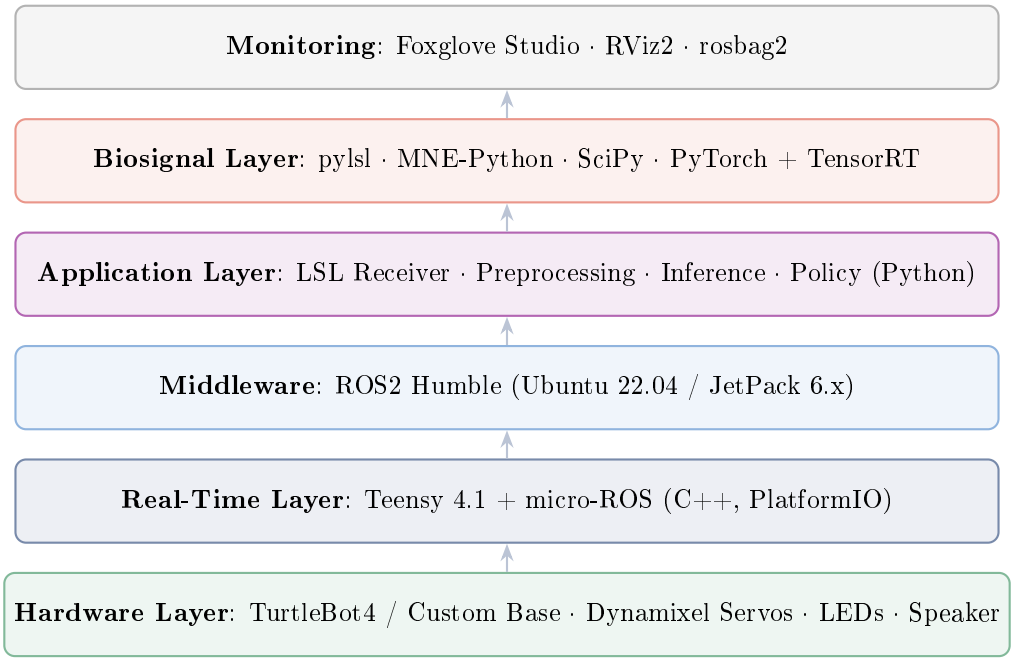

Figure 4: Software stack layers from hardware to monitoring.

5.2 Key Libraries

Table 7: Software libraries by purpose.

Purpose	Library
Biosignal streaming	`pylsl` (Lab Streaming Layer Python bindings)
EEG preprocessing	`MNE-Python` (filtering, artifact rejection, epoching)
Signal processing	`SciPy` (bandpass filters, spectral analysis)
Model inference	`PyTorch` + `torch2trt` or `onnxruntime-gpu`
Behaviour policy	Custom Python; later `Gymnasium` for RL
Robot control	`rclpy` (ROS2 Python), `geometry_msgs`, `Dynamixel SDK`
Teensy firmware	`micro-ROS` Arduino library, `Dynamixel2Arduino`
Orchestration	ROS2 Launch (Python launch files)
Logging	`rosbag2` for recording all ROS2 topics
Visualisation	`RViz2`, `Foxglove Studio` (web-based)

5.3 ROS2 Node Architecture

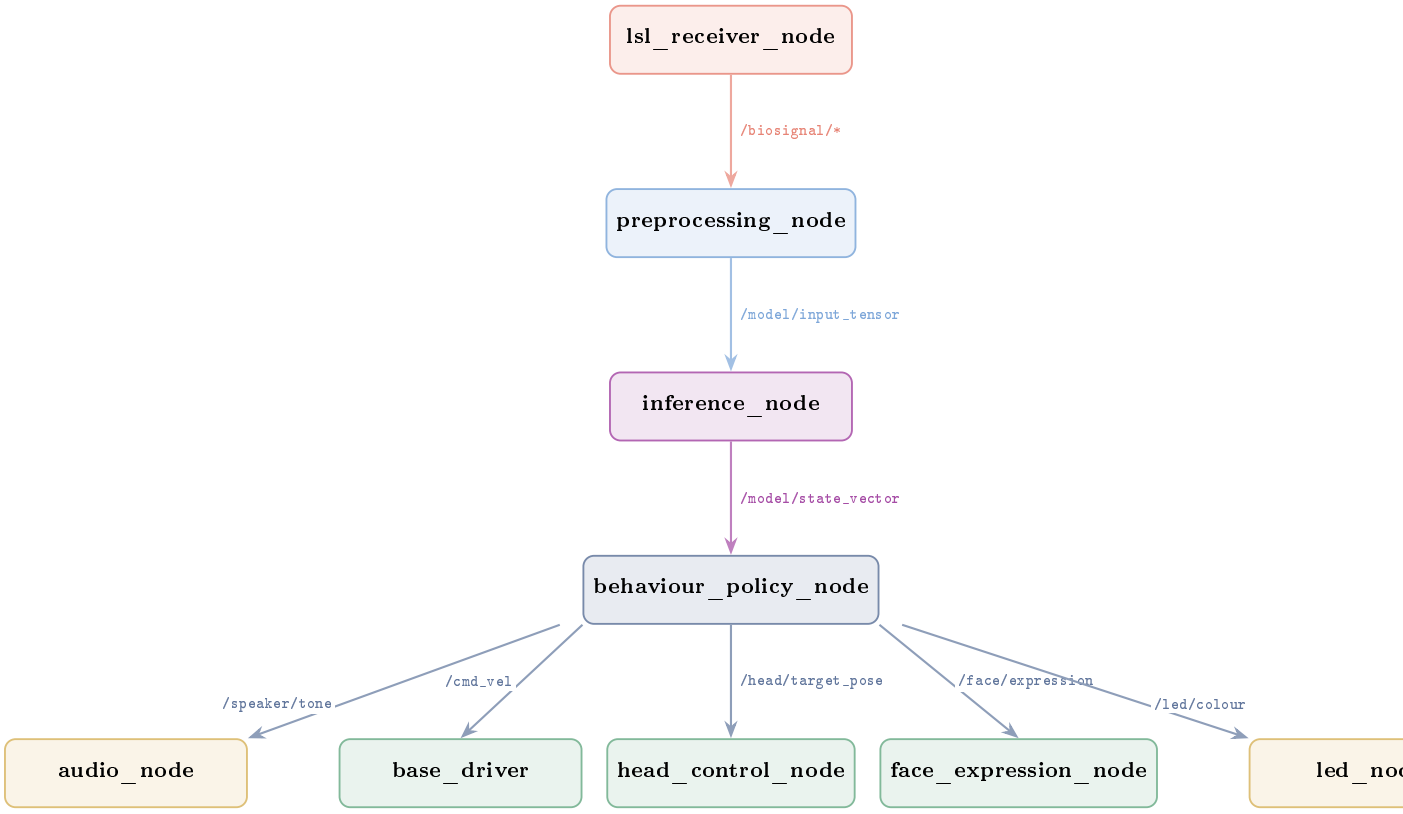

Figure 5: ROS2 node graph showing all nodes and the topics that connect them.

6 Real-Time Data Pipeline

6.1 End-to-End Latency Budget

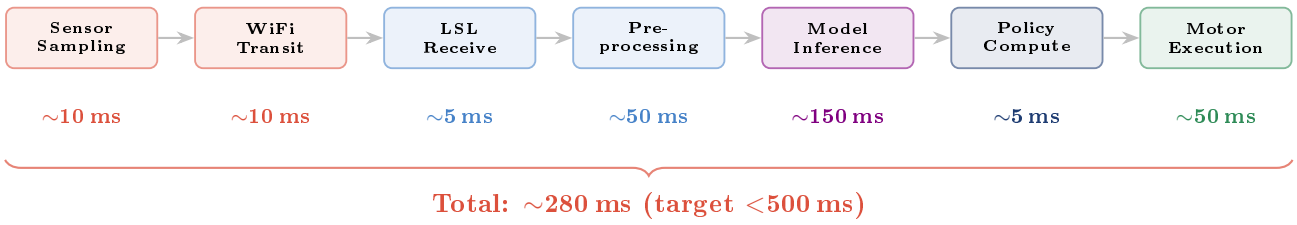

Figure 6: Latency budget across the data pipeline. The dominant cost is model inference.

6.2 Stream Synchronisation

All biosignal streams are transmitted via **Lab Streaming Layer (LSL)**, which provides network-level clock synchronisation across devices. Key considerations:

- **Variable sample rates**: EEG at 250 Hz, fNIRS at 10 Hz, eye tracking at 60–120 Hz, StretchSense at 50 Hz, audio at 16 kHz.
- The **preprocessing node** resamples all streams to a common timebase and windows them into aligned 500 ms segments (configurable), sliding by 100 ms for a 10 Hz output rate.

- **Missing modalities**: the model should handle variable-length input or use zero-masking for absent streams, enabling graceful degradation.

6.3 Signal Processing per Modality

Table 8: Preprocessing steps per biosignal modality.

Modality	Preprocessing Steps
EEG	Bandpass 1–45 Hz, notch filter 50 Hz (mains), artifact rejection (amplitude threshold + ICA), re-referencing, epoch windowing
Eye Tracking	Blink removal/interpolation, fixation detection (I-VT algorithm), pupil baseline normalisation, saccade extraction
fNIRS	Modified Beer-Lambert Law (raw intensity → HbO/HbR concentrations), bandpass 0.01–0.2 Hz, motion artifact correction
StretchSense	Calibration offset removal, first derivative for movement velocity, smoothing
Audio	Voice Activity Detection (VAD), pitch extraction (CREPE or pYIN), energy envelope, spectral features for prosody

7 Behaviour Design

7.1 State-to-Behaviour Mapping

The behaviour policy maps the model's output state vector to robot actions. Figure 7 illustrates the mapping across key state dimensions.

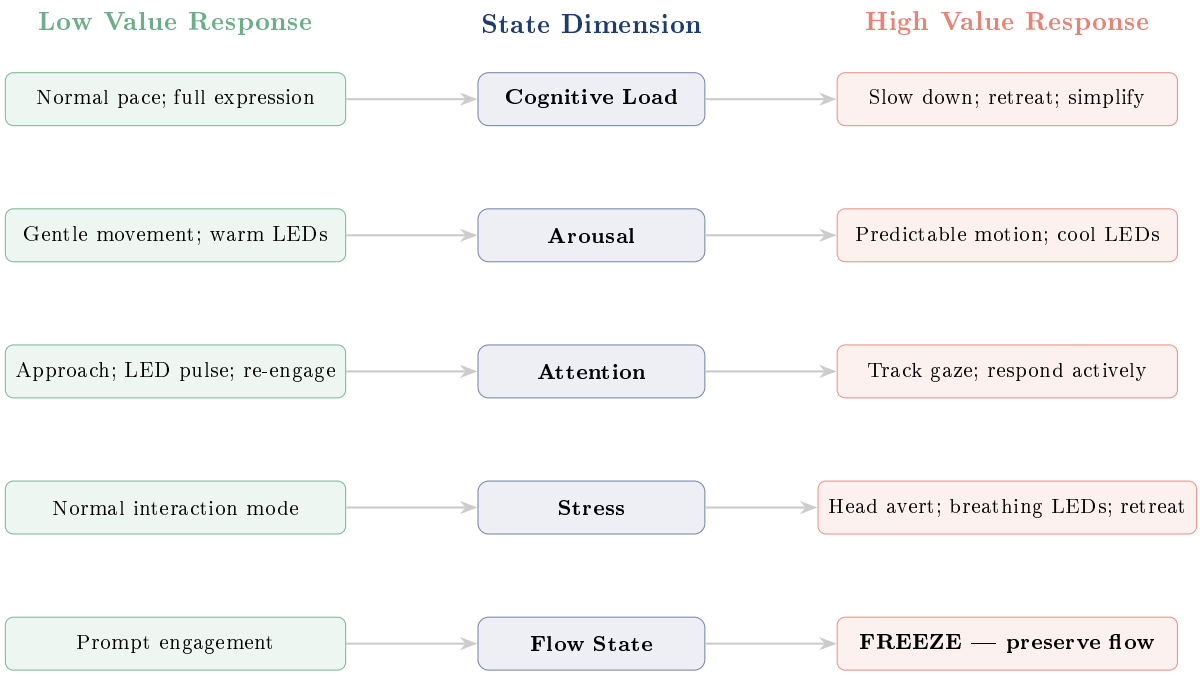

Figure 7: Behaviour mapping: each neurobehavioural state dimension maps to distinct robot responses at low and high values. Flow state detection has highest priority and overrides all other behaviours.

7.2 Behaviour Primitives

The robot maintains a library of atomic behaviour primitives that the policy selects and blends:

Listing 1: Behaviour primitives (Python, v1 rule-based).

```python
class BehaviourPrimitives:
    def retreat(self, distance=0.3):
        """Move base backward -- give space when overloaded."""

    def approach(self, distance=0.2):
        """Move base forward -- re-engage when attention drops."""

    def slow_breathing_led(self, period=4.0, colour=(0, 0, 80)):
        """Slow pulsing LED -- calming during high stress."""

    def alert_pulse_led(self, colour=(255, 180, 0)):
        """Quick gentle pulse -- attention bid."""

    def set_expression(self, expression: str):
        """Set face: 'neutral','attentive','calm','encouraging'"""

    def head_track(self, target_position):
        """Pan-tilt head to track human -- active engagement."""

    def head_avert(self):
        """Look away -- reduce pressure during overload."""

    def freeze(self):
        """Stop all movement -- preserve flow state."""

    def play_tone(self, tone: str):
        """Soft chime for attention bids, confirmations."""
```

7.3 Policy Logic

Listing 2: Priority-based behaviour policy (v1).

```python
def compute_action(state_vector, actions):
    cog_load = state_vector['cognitive_load']  # 0-1
    arousal  = state_vector['arousal']         # 0-1
    attention = state_vector['attention']      # 0-1
    stress   = state_vector['stress']          # 0-1
    flow     = state_vector['flow_state']      # 0-1

    # Priority 1: Preserve flow (override everything)
    if flow > 0.7:
        return actions.freeze()

    # Priority 2: Stress response
    if stress > 0.7:
        return actions.compose(
            actions.head_avert(),
            actions.slow_breathing_led(),
            actions.retreat(0.2),
            actions.set_expression('calm'))

    # Priority 3: Cognitive overload
```

```
    if cog_load > 0.75:
        return actions.compose(
            actions.set_expression('calm'),
            actions.retreat(0.15),
            actions.slow_breathing_led(period=5.0))

    # Priority 4: Re-engagement
    if attention < 0.3:
        return actions.compose(
            actions.approach(0.1),
            actions.alert_pulse_led(),
            actions.set_expression('attentive'))

    # Default: normal engaged interaction
    return actions.compose(
        actions.head_track(human_position),
        actions.set_expression('attentive'))
```

Note

Use **smooth interpolation** between behaviour states, not hard switches. Add small randomised micro-movements (gentle head sways, slow LED colour drift) to make the robot feel alive and avoid the uncanny valley of rigid state machines.

8 Build Order and Milestones

Figure 8 shows the phased build timeline. Each phase produces a demonstrable deliverable.

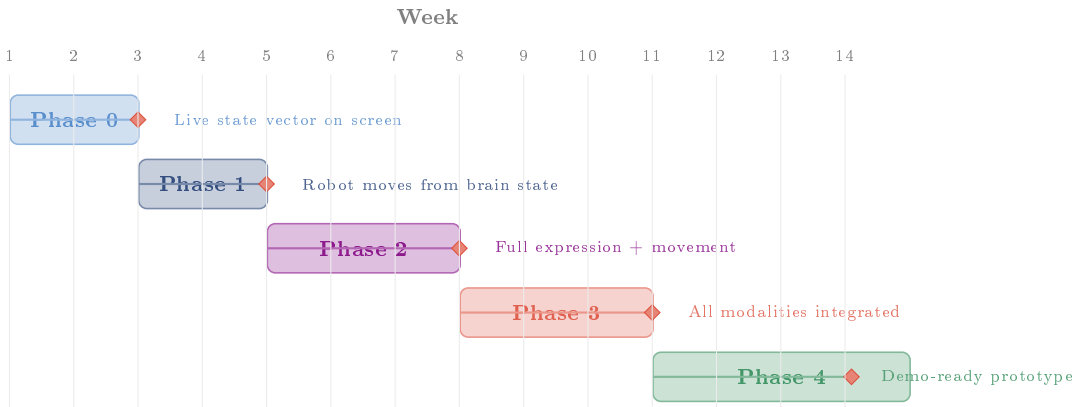

Figure 8: Build timeline across 14 weeks with five phases. Each diamond marks a demonstrable milestone.

8.1 Phase 0: Software Foundations (Weeks 1–2)

Deliverable: Human wears EEG, state vector updates live on screen

- Set up ROS2 Humble on Ubuntu 22.04 (development laptop)
- Write `lsl_receiver_node`—connect to one real LSL stream (e.g. EEG headset)
- Write `preprocessing_node`—basic bandpass filter, publish processed data
- Write `inference_node`—load foundation model in PyTorch, run inference on windowed data
- Verify end-to-end: EEG → LSL → preprocessing → model → state vector on terminal
- Define custom ROS2 message types (`NeuroBehaviouralState.msg`)

8.2 Phase 1: Robot Base (Weeks 3–4)

Deliverable: Robot physically moves in response to human brain state

- Acquire and bring up TurtleBot4 Lite in ROS2
- Verify `/cmd_vel` control (keyboard teleop first)
- Write `behaviour_policy_node`—map state vector to approach/retreat
- Connect full loop: human state → robot movement
- Set up Jetson Orin NX, flash JetPack, install ROS2 + PyTorch
- Migrate all nodes from laptop to Jetson

8.3 Phase 2: Expressive Head (Weeks 5–7)

Deliverable: Robot moves, looks at you, changes expression and LEDs based on state

- Design and 3D-print head housing
- Mount Dynamixel pan-tilt + round display
- Write `face_expression_node`—animated eyes (pygame or lightweight OpenGL)
- Write `head_control_node`—pan-tilt tracking
- Mount LED ring, write `led_node`
- Add speaker + amplifier, write `audio_node`
- Expand behaviour policy to use all output channels

8.4 Phase 3: Multi-Modal Input (Weeks 8–10)

Deliverable: Full multimodal input, richer state vector, nuanced behaviour

- Add eye tracking, fNIRS, StretchSense, and microphone to LSL pipeline
- Update `preprocessing_node` for each modality
- Handle variable sample rates: resampling and temporal alignment
- Retune windowing parameters
- Validate state vector quality with all streams active

8.5 Phase 4: Polish and Optimise (Weeks 11–14)

Deliverable: Robust, low-latency, demo-ready prototype

- Convert model to TensorRT, benchmark on Jetson
- Profile and optimise full pipeline latency
- Add `rosbag2` recording for experiment logging
- Add Foxglove Studio dashboard for live monitoring
- Tune behaviour policy with real users (iterative)
- Safety features: e-stop button, collision avoidance
- Design and print final housing

9 Risks and Mitigations

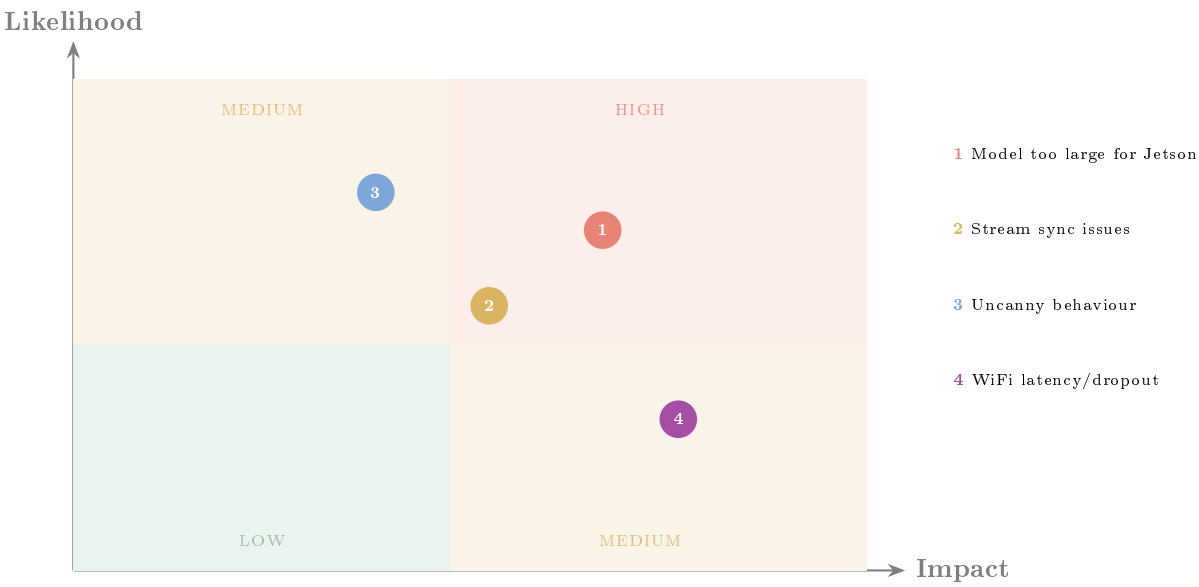

Figure 9: Risk matrix. Numbered risks are detailed in the mitigations below.

Risk 1: Model Too Large for Jetson

Mitigation: Design the inference node to support both local and remote backends from day one. Start with server offload (WiFi + gRPC), optimise with TensorRT/quantisation, and migrate to Jetson once the model fits. A gRPC round-trip on local 5 GHz WiFi adds only ∼20–50 ms.

Risk 2: Multi-Stream Synchronisation

Mitigation: LSL handles clock sync well, but test with all streams simultaneously early in Phase 3. Variable sample rates (EEG 250 Hz, fNIRS 10 Hz, eye tracking 60–120 Hz) require explicit resampling and temporal alignment in the preprocessing node. Build alignment unit tests before integrating.

⚠ Risk 3: Uncanny / Robotic Behaviour

Mitigation: Use smooth interpolation between behaviour states, not hard threshold switches. Add randomised micro-movements (small head sways, gentle LED colour drift) to create liveliness. Tune transition speeds empirically with real users—budget significant time in Phase 4 for this.

⚠ Risk 4: WiFi Latency or Dropout

Mitigation: Use a **dedicated 5 GHz WiFi router** (not shared network) travelling with the setup. Consider a USB WiFi dongle for the most critical low-latency stream (e.g. EEG) if standard WiFi proves unreliable. Implement a brief state-hold buffer so the robot maintains its last known state during transient dropouts.

10 Data Clarification

⚠ Important: Input Modality Mismatch

The foundation model specification states that inference takes **EEG, ECG, and IMU** as inputs. However, the recorded data description explicitly states **no ECG** and **no IMU** (StretchSense measures deformation, not inertial data) in either recording configuration.

This must be resolved before wiring the pipeline:

- If the model genuinely requires ECG and IMU: source a wearable ECG (e.g. Polar H10, BLE + LSL bridge) and IMU (e.g. Movella DOT) for the human to wear during operation.
- If the model was trained on the recorded modalities (EEG, eye tracking, fNIRS, StretchSense, audio): update the inference specification to match.

11 Future Directions

1. **Learned Behaviour Policy (Phase 5)**: Replace the rule-based policy with a reinforcement learning agent. Use recorded `rosbag2` data and human feedback to train the policy. The foundation model's state embeddings become the RL observation space.
2. **Manipulation**: Add a single robot arm (e.g. Trossen PincherX 100) for handing objects, pointing, or gestural communication.
3. **Multi-Person Interaction**: Extend the system to read biosignals from multiple participants simultaneously and manage group interaction dynamics.
4. **Longitudinal Adaptation**: Store state history per individual and adapt the robot's baseline behaviour over repeated interactions.
5. **Clinical Applications**: Deploy the platform in therapeutic contexts (e.g. stress management, attention training, cognitive rehabilitation) with clinician-designed behaviour protocols.

A Communication Topology

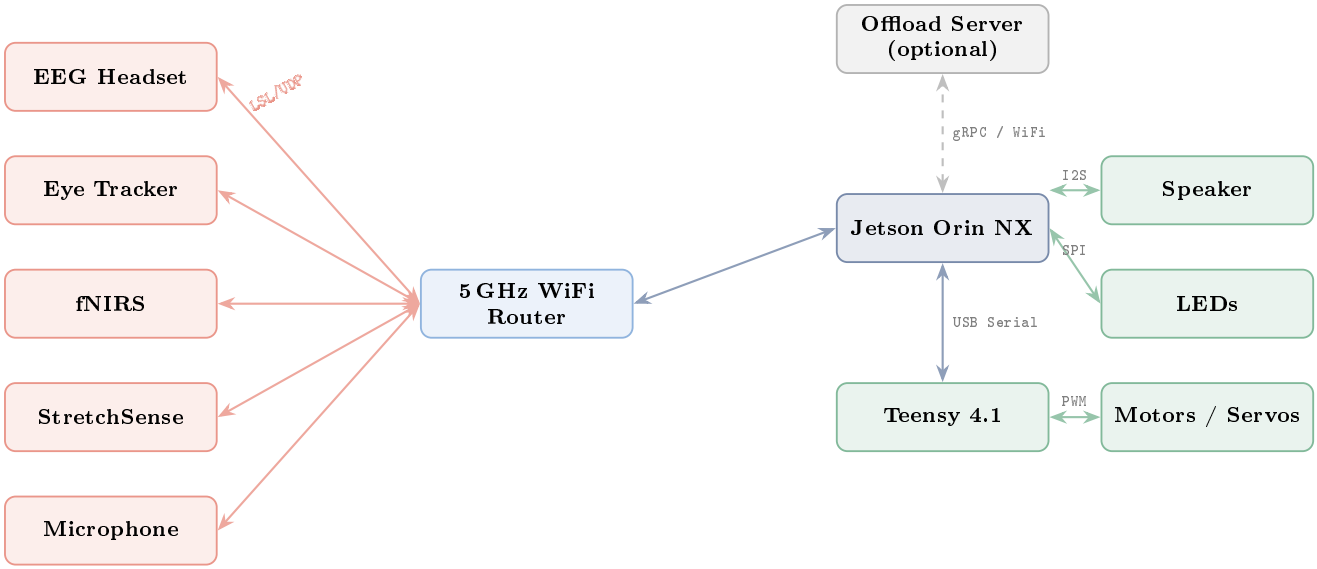

Figure 10: Full communication topology showing all physical and network links.